

Project 1: Hello Django - Lab Manual

Goal: Build a very basic Django website with three pages: Home, About, and Contact. This project avoids database, forms, login, and models so students can first become confident with Django fundamentals.

What Students Will Learn

- Django project and app structure
- How URL routing works
- How a URL calls a view function
- How a view renders an HTML template
- How to use template inheritance with base.html
- How to navigate between pages using Django URL names

Recommended Coding Workflow

Step	What to Code	Why This Step Matters
1	Create Django project	Creates the main configuration folder and manage.py
2	Create pages app	Keeps page-related code separate from project settings
3	Register app in settings.py	Allows Django to recognize the app
4	Create views	Each view prepares a response for a page
5	Create URL patterns	Maps browser URLs to view functions
6	Create templates	Stores the HTML files shown to users
7	Add base.html	Avoids repeating common layout and navigation
8	Test every page	Confirms that routing, views, and templates work together

Step 1: Create the Project

Command:
`django-admin startproject hello_django`
`cd hello_django`

Explanation:
This creates the main Django project. The manage.py file is used to run Django commands. The inner hello_dj

Step 2: Create the App

Command:
`python manage.py startapp pages`

Explanation:
The pages app will contain the view functions and templates for Home, About, and Contact pages.

Step 3: Register the App

File: hello_django/settings.py

Code:
`INSTALLED_APPS = [`
`...`

```
    'pages',  
]
```

Explanation:

Django will not use the app unless it is registered inside `INSTALLED_APPS`.

Step 4: Create View Functions

File: `pages/views.py`

Code:

```
from django.shortcuts import render  
  
def home(request):  
    return render(request, 'pages/home.html')  
  
def about(request):  
    return render(request, 'pages/about.html')  
  
def contact(request):  
    return render(request, 'pages/contact.html')
```

Explanation:

A view function receives a request and returns a response. Here each function returns an HTML template.

Step 5: Connect URLs to Views

File: `hello_django/urls.py`

Code:

```
from django.contrib import admin  
from django.urls import path  
from pages import views  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', views.home, name='home-page'),  
    path('about/', views.about, name='about-page'),  
    path('contact/', views.contact, name='contact-page'),  
]
```

Explanation:

When a user visits `/about/`, Django runs the `about` view. URL names help us link pages safely from templates.

Step 6: Create Templates

Folder:

`pages/templates/pages/`

Files:

```
base.html  
home.html  
about.html  
contact.html
```

Explanation:

Templates contain the HTML shown in the browser. We keep them inside the app so Django can find them automatically.

Step 7: Use Template Inheritance

File: `pages/templates/pages/base.html`

Explanation:

`base.html` contains common layout such as the navbar, CSS, and page structure. Other pages extend it so we can reuse the common layout.

Step 8: Run and Test

Command:

```
python manage.py runserver
```

Open:

```
http://127.0.0.1:8000/
```

```
http://127.0.0.1:8000/about/
```

```
http://127.0.0.1:8000/contact/
```

Explanation:

Test every page after coding. Students should learn to build one small part, run the server, check output,

Classroom Teaching Notes

Suggested explanation order:

1. Browser sends request to a URL.
2. urls.py checks which path matches.
3. The matched path calls a view function.
4. The view function renders a template.
5. The browser displays the final HTML.

Simple request-response flow:

Browser URL -> urls.py -> views.py -> template HTML -> browser output

Mini exercises for students:

- Change the Home page heading.
- Add a Services page.
- Add a new URL for /services/.
- Add a link to Services in the navbar.
- Change the background color in base.html.